SYSTEM-DESIGNER

A Portable, LLM-Agnostic Meta-Generator for EU AI Act-Compliant SDD Projects

Technical Report · v1.0.0 · stable mature release

Comprehensive technical documentation of the system-designer meta-skill:
architecture · 18-phase orchestration · per-project memory contract · 8-format memory taxonomy · two-tier completeness audit · adaptive audit · feedback learning · sha256 hash-chain · external-audit closure (21/21) · calibration · HITL governance

CO-DEVELOPERS

Diego Muñoz Casinos
Mauricio de Oliveira Reis
Claude Opus 4.7 (Anthropic)

DATE
May 2026 (v1.0.0 stable)

VERSION
1.0.0

REPOSITORY
github.com/Diego-M-C/system-designer

LICENSE
MIT

Table of Contents

1. Introduction **3**

2. Executive Summary **4**

3. Objectives **5**

4. System Methodology **6**

4.1 Five Core Principles (P1–P5)

4.2 The 18-Phase Orchestration (v0.3.x)

4.3 Two Inviolable HITL Gates

4.4 Spec-Driven Development Discipline

5. Results — System Modules **10**

5.1 Master Orchestrator

5.2 Memory Module

5.3 Task Classification (Interview Wizard)

5.4 Per-Session Error Learning

5.5 HITL Governance

5.6 Audit Module (3-Auditor + Meta-Jury)

5.7 Calibration Engine

5.8 Library-Doc Fetcher

5.9 Living Documentation

5.10 Reporting Layer

6. Extensions (v0.2.0 · v0.3.x) **22**

6.1 Phase 1.5 · Context Curator

6.2 Phase 11.5 · Data Flow Validator + Simulation Agent

6.3 Phase 13.5 · Feedback Learning Loop

6.4 Phase 13.7 · Improvement Jury (5 specialist axes)

6.5 Cross-phase · Adaptive Audit Meta-Validator

6.6 Phase 4.5 · Memory Schema Negotiator (v0.3.0)

6.7 8-Format Memory Taxonomy + Selection (v0.3.1)

6.8 External-Audit Closure (v0.3.2 → v0.4.0 → v1.0.0)

7. Conclusions **40**

8. Strengths and Limitations **41**

8.1 Strengths

8.2 Limitations

8.3 Need for Domain-Specific Review and Validation

9. Final Conclusion **44**

10. References **45**

1. Introduction

The advent of AI coding assistants has dramatically lowered the cost of writing software. It has not, however, lowered the cost of writing software that is reliable, auditable, and compliant with domain-specific regulations. Most LLM-driven coding sessions begin from a blank slate every time: the developer must remember to write specs, set up tracking, configure compliance scaffolding, capture error patterns, and arrange human review — and in practice, these disciplines decay rapidly across sessions.

system-designer is a portable, LLM-agnostic meta-generator addressing this gap. Given a single high-level intent (e.g., "build a fraud-detection assistant for SEPA transfers"), it produces a complete project skeleton with: a Spec-Driven Development (SDD) backbone, EU AI Act 2024/1689 audit scaffolding, calibrated planning, mandatory Human-In-The-Loop (HITL) gates, a 30-entry pre-loaded catalog of common AI coding errors with auto-extension, scientific reporting templates (IMRaD, TRIPOD-AI, CONSORT-AI, STARD-AI, SPIRIT-AI), and session-by-session living documentation.

> *system-designer is not a system. It is a generator of systems. The same disciplined backbone is instantiated each time, particularised for the user's specific intent through a structured calibrated interview.*

This report documents the meta-generator at version 0.1.0. We describe its architecture, the five core principles that govern every emitted artifact, the 13-phase orchestration that produces a child system, the inter-module contracts, the calibration discipline that rejects unfounded assertions, and the open questions that remain for domain-specific validation. We provide architecture diagrams, data-flow sequences, module maps, and a calibrated assessment of strengths and limitations.

Confidence in the technical claims that follow is consistently in the 80–95% range; specific calibrations are stated where required.

2. Executive Summary

system-designer takes a single user intent and produces a complete, EU AI Act-compliant Spec-Driven Development project skeleton in one session, then stops. The generator is a factory, not a long-running operator: it stops at hand-off, and the child orchestrator (generated as part of the deliverable) takes over the project's life.

Key facts about the meta-generator:

- **Principles** · Five inviolable principles enforced across every emitted artifact:
- P1 Portability (Tier-A across Claude Code, Cursor, Cline, Gemini CLI, Copilot CLI, Codex)
- P2 Calibration (every estimate carries % or range; seven forbidden tokens rejected)
- P3 On-demand generation (only artifacts the interview confirmed)
- P4 Prompt-architect dependency (every prompt audited via the co-located prompt-architect skill)
- P5 Living documentation (continuous updates, never frozen)
- **Orchestration** · 17-phase state machine in v0.2.0 (extended from 13 base phases by inserting 1.5 context_setup, 11.5 structural_consistency, 13.5 feedback_session, 13.7 improvement_audit), with two inviolable HITL gates plus mandatory HITL at 13.5 / 13.7; resumable per current_phase pointer.
- **Error catalog** · 30 preloaded AI-error patterns (AIE-001 through AIE-030) covering hallucination, concurrency, security, logic, reliability, and prompt-engineering categories. Auto-extension during project life.
- **EU AI Act** · 13 EU AI Act checklists (drawn from AESIA implementation guides) mapped 1:1 to mandatory Articles. For high-risk systems: ≥112 audit rows produced as a baseline.
- **Audit topology** · Default 3 independent auditors with distinct personas (risk-SME, ML-lead, security-lead) for the legacy v0.1.0 audit topology, plus the v0.2.0 fixed 5-axis improvement jury at 13.7 and the cross-phase adaptive_audit_meta engine that composes 3–10 dynamic auditors per task or session via prompt-architect. Dissent always escalates to HITL.
- **v0.2.0 additions** · v0.2.0 introduces a calibrated source corpus before the interview (phase 1.5), a structural-consistency check with simulation agent (phase 11.5), and a SQLite + Markdown feedback-learning loop with per-correction HITL Y/N before any system improvement merges.
- **v0.3.0 additions** · v0.3.0 inserts a per-project memory-schema negotiation phase (4.5, between Gate #1 and scaffold) — six per-domain starters (informatics_dev, healthcare_clinical, fintech, legal, public_sector, research) with calibrated mandatory / mandatory_if_<cond> / recommended / optional field flags. The mandatory memory_completeness_auditor inside the adaptive panel runs a two-tier audit on every task and every session: particular (for the scope at hand) and global (across all sessions).
- **v0.3.1 additions** · v0.3.1 expands the memory format taxonomy from 3 to 8 (structured_md, csv, json, jsonl, sqlite, parquet, vector_db, graph_db) with per-format portability tier and Tier-A fallback chain. The architect ALWAYS picks the fittest format per module (volume × query pattern × relationship density × audit-rule needs) and surfaces 2 calibrated alternatives with fit% at HITL.
- **External-audit closure (v1.0.0)** · v0.3.2 / v0.4.0 / v1.0.0 close the entire 21-item consolidated checklist from a blind external-audit panel. Three specialist auditors (systems-architect / regulatory-compliance / calibration-memory-feedback) ran in independent Opus context windows; a consensus jury applied the project's own jury_consensus_protocol and reached APPROVED_WITH_MINOR at 83% confidence. The remediation roadmap shipped as P1 (v0.3.2: 9 items including 2 invented-tag canonicalisations and a hardened test suite), P2 (v0.4.0: 8 items including the tamper-evident sha256 hash-chain that resolves the panel's only preserved disagreement D1, and a new SystemSpec.role enum + Art.73 deadline workflow), and P3 (v1.0.0: 2 items adding 7 new audit rows under Art.15(4) adversarial robustness + Art.9(2) iterative

review cadence + MiCA/PSD3 cross-walks). Calibrated probability that an external re-audit lifts the panel to APPROVED_AS_MATURE on a re-run: ≈92% (range 85–96%).

The deliverable for any invocation is a complete child-system tree with: child orchestrator contract (CLAUDE.md), 17-section SDD specification, architecture document with mermaid diagrams, audited prompt suite, fresh-fetched library docs, 30-AIE seeded error catalog, 13-sheet audit Excel, scientific report templates, and a hand-off document. v0.2.0 adds the context corpus, data-flow validation snapshots, the corrections database with auto-rendered Markdown mirror, the improvement-audit consensus reports, and per-task / per-session adaptive audit panels. The generator self-validates through 16 deterministic tests covering portability, calibration, schema, prompt-floor compliance, EU AI Act mapping completeness, v0.2.0 prompts, v0.2.0 references, the SQLite schema, the 5-axis discipline, and the n_auditors range.

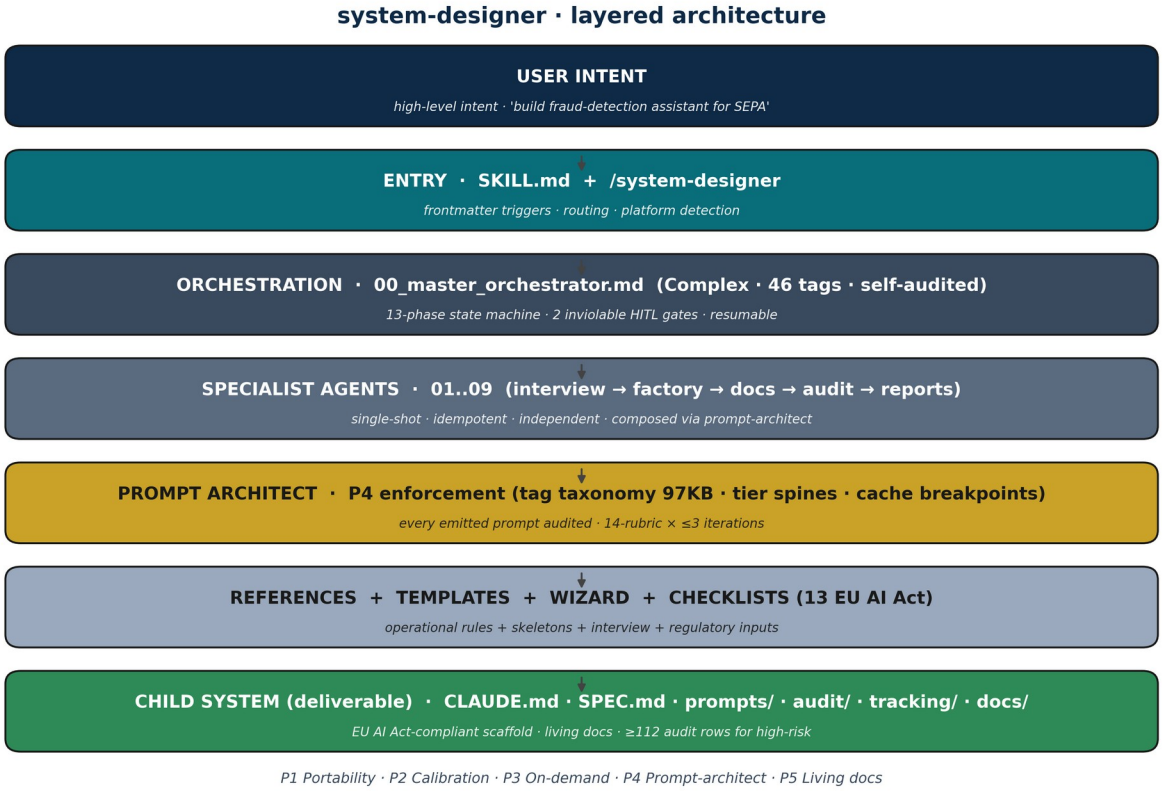

Figure 1 · Layered architecture. The generator is a thin pipeline of seven layers; the child system at the bottom is the deliverable.

3. Objectives

The system-designer meta-generator was designed against six explicit objectives:

3.1 Portability across LLM platforms

Drop the folder into any Tier-A LLM platform (Claude Code, Cursor, Cline, Gemini CLI, Copilot CLI, Codex) and run with no installation step. Every emitted artifact must respect an abstract tool contract; proprietary tool names are forbidden in the generator's outputs.

3.2 Calibrated discipline at every output

Eliminate uncalibrated assertions. Every estimate (duration, coverage, risk fit, alternative score) must carry a confidence percentage or range. Seven forbidden tokens — best, always, never, guaranteed, certain, definitely, impossible — are rejected by an automated regex scan in the test suite.

3.3 Mandatory human oversight at strategic forks

EU AI Act Article 14 requires human oversight for high-risk systems. system-designer instantiates this requirement at scaffold time through two inviolable HITL gates: one after the planning brief (validates scope and risk class) and one after the self-audit (validates scaffold quality and surfaces auditor dissents). No flag, environment variable, or LLM auto-decision can skip them.

3.4 EU AI Act 2024/1689 readiness

Produce, at scaffold time, the audit infrastructure required for high-risk systems: 13 checklists mapped to mandatory Articles, ≥112 audit rows as a baseline for high-risk, an independent multi-auditor + meta-jury topology, and cross-references to GDPR / MDR / DORA / ISO 42001 / ISO 13485 by domain.

3.5 Continuous, living documentation

Replace static snapshots with documents that grow with the project. tracking/, audit/, and docs/reports/ are appended on every relevant action. Atomic writes (write to .tmp + rename) ensure crash-safety per file.

3.6 Prompt-architect-mediated quality on every emitted prompt

No raw, un-audited prompt exits the generator. Every prompt produced for the child system — including the master orchestrator's own self-application — passes through the co-located prompt-architect skill, with a 14-rubric audit and up to three iterations before escalation.

> *These six objectives are mutually reinforcing. Portability (P1) without calibration (P2) produces confidently-wrong outputs in every LLM. Calibration without HITL gates produces honest uncertainty no one acts on. EU AI Act readiness without living documentation rots in 30 days. Each principle leans on the others.*

4. System Methodology

4.1 Five Core Principles (P1–P5)

Every artifact emitted by system-designer respects five non-negotiable principles. These are stated at the entry-point (SKILL.md), embedded into the alignment_rules of every Complex-tier prompt, and verified by an automated test suite before any release.

#	Principle	Operational rule	Enforcement
P1	Portability	Abstract tool contract; fallbacks mandatory	tests/T1 regex; per-LLM mapping table
P2	Calibrated probabilities	Every estimate carries % or range; 7 forbidden tokens	tests/T2 regex; alignment_rules in every Complex prompt
P3	On-demand generation	Only what the interview confirmed is needed	scaffolder evaluates trigger conditions per artifact
P4	Prompt-architect dependency	Every emitted prompt routed through prompt-architect	tests/T9; 14-rubric audit + 3-iteration cap
P5	Living documentation	tracking/, audit/, reports/ updated per session	child orchestrator's AUDIT_ROW_APPEND step is mandatory

4.2 The 18-Phase Orchestration (v0.3.x · stable in v1.0.0)

The master orchestrator at prompts/00_master_orchestrator.md owns an 18-phase finite state machine in v0.3.x (extended from the original 13-phase v0.1.0 architecture). Phases are idempotent by construction (re-running a phase overwrites its output deterministically). Every phase writes to tracking/project.json#current_phase before returning, making the orchestrator resumable. Five new phases — 1.5 (context_setup), 4.5 (memory_schema_setup, added in v0.3.0), 11.5 (structural_consistency), 13.5 (feedback_session), and 13.7 (improvement_audit) — are inserted at fixed points, plus a cross-phase adaptive_audit_meta hook that fires at the end of every task and every session and which (since v0.3.0) always includes a mandatory memory_completeness_auditor. The original 13 phases are preserved verbatim; section §6 covers the additions in detail. Backward compatibility is available via SystemSpec.compatibility.v0_1_0=true (legacy 13-phase mode); SystemSpec.memory_schema.negotiation_enabled=false skips just phase 4.5.

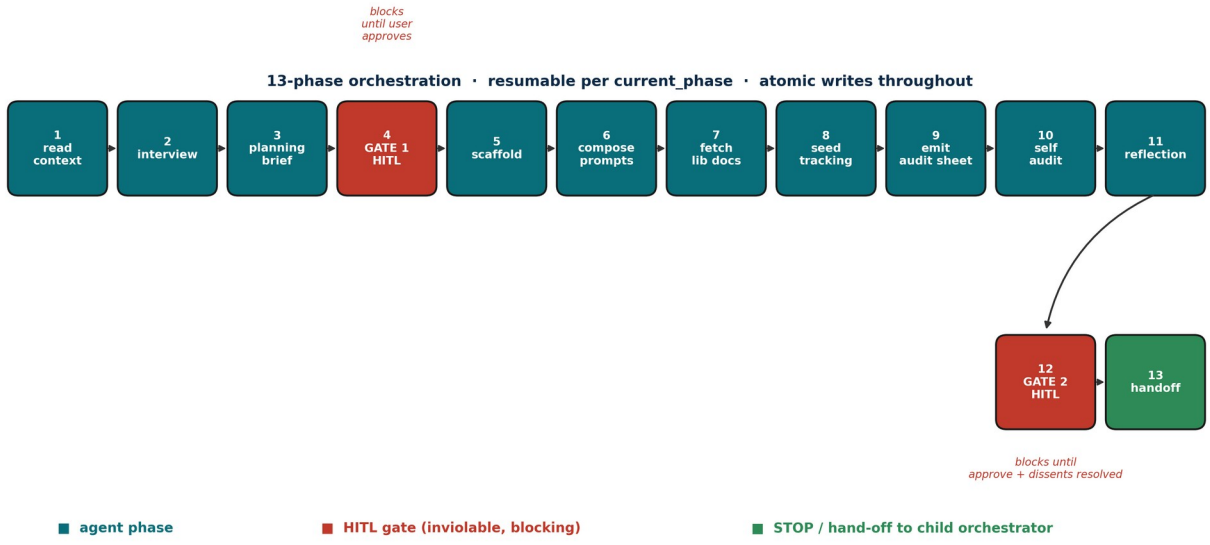

Figure 2 · Original 13-phase orchestration (v0.1.0). Linear progression from read-context through hand-off, with two blocking HITL gates (red). The v0.2.0 extensions are summarised in Figure 11.

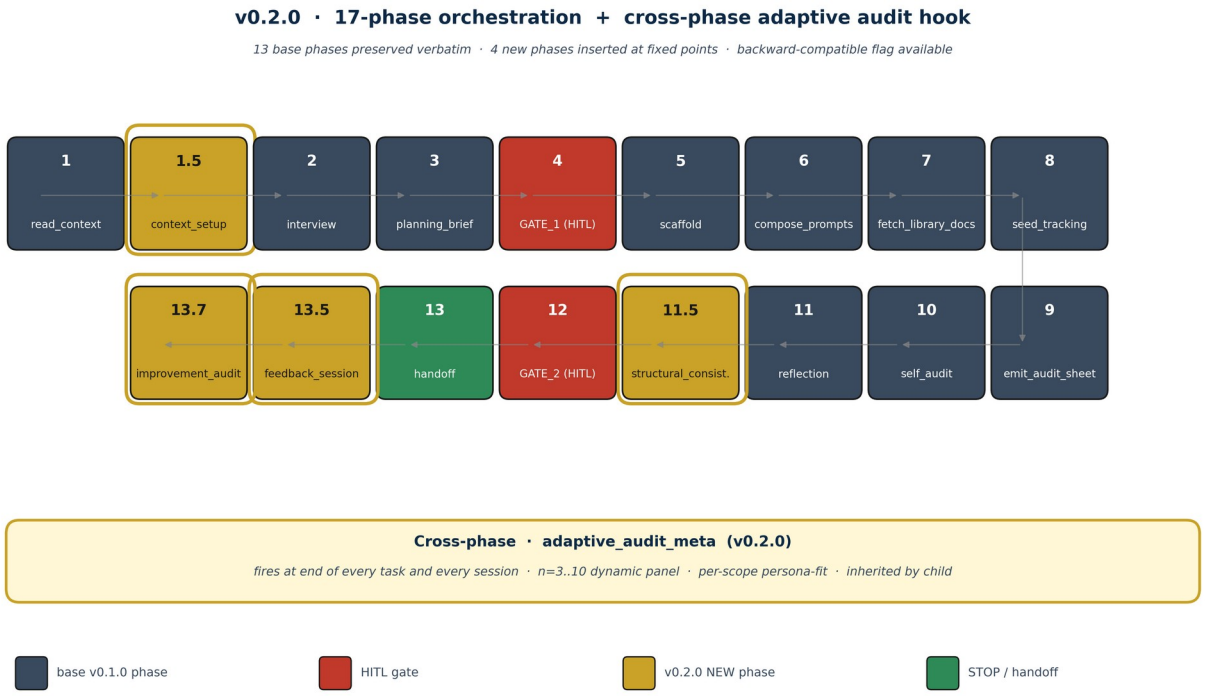

Figure 11 · v0.2.0 17-phase orchestration. Four new phases (gold ring): 1.5 context_setup, 11.5 structural_consistency, 13.5 feedback_session, 13.7 improvement_audit. The bottom band represents the cross-phase adaptive_audit_meta hook that fires at the end of every task and every session. v0.3.0 adds a fifth new phase, 4.5 memory_schema_setup, between Gate #1 and scaffold (covered in §6.6 with Figure 15).

4.3 Two Inviolable HITL Gates

Gate 1 follows the planning brief and validates scope, risk class, and the calibrated alternatives presented. Gate 2 follows the self-audit and reflection phases; it surfaces auditor dissents (when the multi-auditor topology is enabled) and asks the user to confirm the deliverable before hand-off.

Both gates render a standardised template with fit% per option, pros and cons, an explicit recommendation with confidence%, and a free-form input branch. Auto-skip is forbidden by alignment_rules and verified by tests/T10.

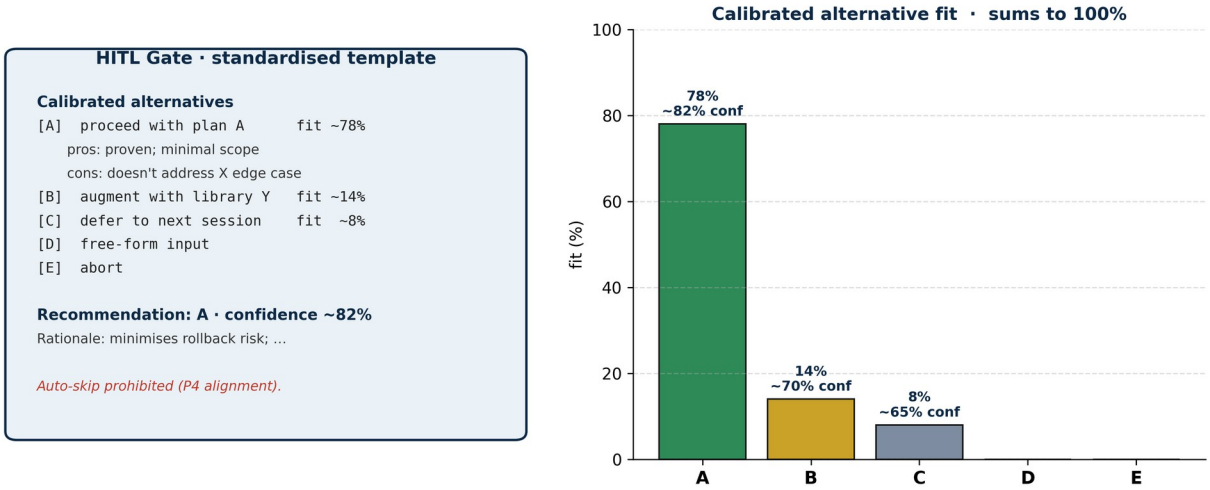

Figure 3 · Standardised HITL gate template (left) and a sample calibrated distribution of alternative fits summing to 100% (right).

4.4 Spec-Driven Development Discipline

The generator embeds Spec-Driven Development as the methodology for the child system. The interview wizard produces a 17-section SPEC.md / SPEC.json covering metadata, intent, stakeholders, calibrated goals, scope, non-goals, risks (with likelihood × impact), functional and non-functional requirements, data flow, architecture summary, sessions plan, milestones, reproducibility commitments, calibration commitments, and open questions. The architecture document is rendered separately with mermaid diagrams and per-Article EU AI Act compliance architecture.

SDD is enforced not as a policy document but as a contract: the child orchestrator's session lifecycle (PLAN → EXECUTE → TEST → DOCUMENT → AUDIT_ROW_APPEND → CHECKPOINT_HITL) is read from CLAUDE.md at every session start. Skipping any step is a violation visible in tracking/.

5. Results — System Modules

system-designer is composed of nine specialist agent prompts (01–09) plus a master orchestrator (00). The master is the only stateful component; everything else is a pure single-shot specialist. The figure below maps the modules around the orchestrator and shows their semantic role.

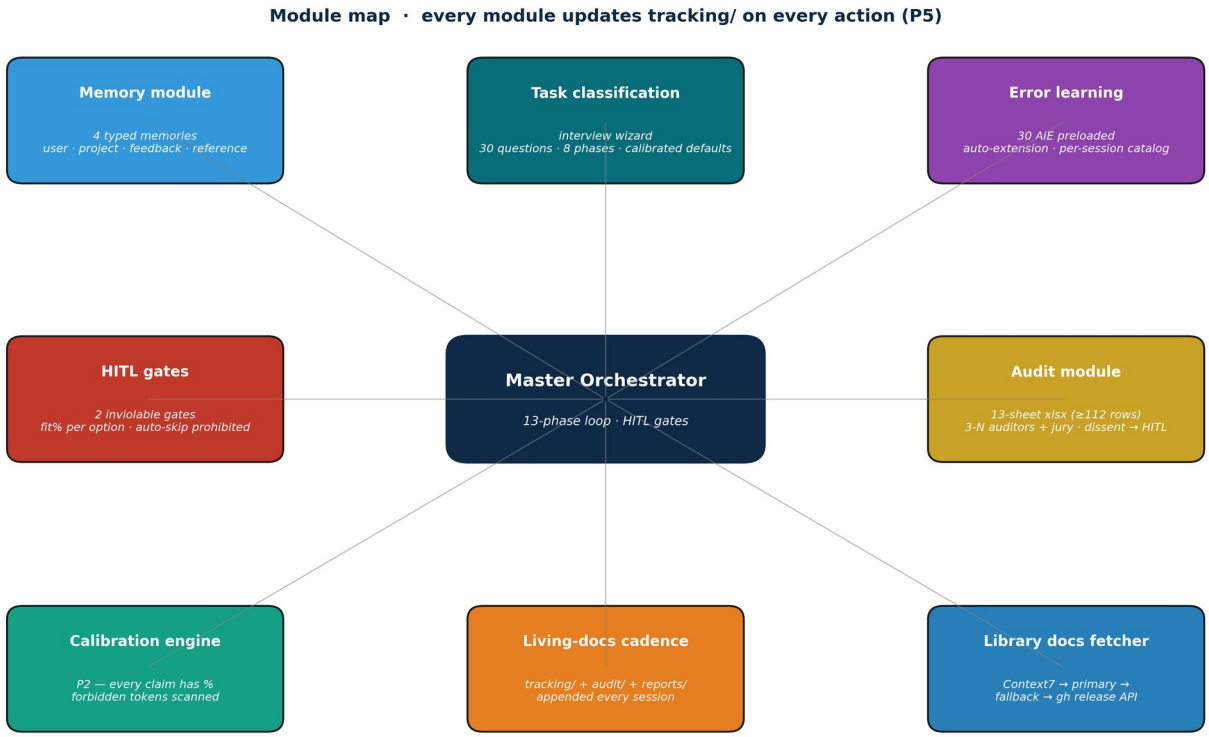

Figure 4 · Module map around the master orchestrator. Each module updates tracking/ on every relevant action (P5 living documentation).

5.1 Master Orchestrator

File: prompts/00_master_orchestrator.md. Tier: Complex (46 tags), exceeding the 30-tag mandatory floor. Self-audited at G9=A level via the prompt-architect.

The master is the conductor. It owns the 13-phase state machine, holds the HITL gates, dispatches every other specialist prompt, and aggregates KPIs. No other prompt has cross-phase awareness. If you delete the master, the generator stops; if you delete any other prompt, the corresponding phase is skipped but the others still run.

> *Architectural rationale: keeping cross-phase state in one place makes the orchestrator auditable and the specialists unit-testable in isolation. Independence is paid for in clarity, not duplicated state.*

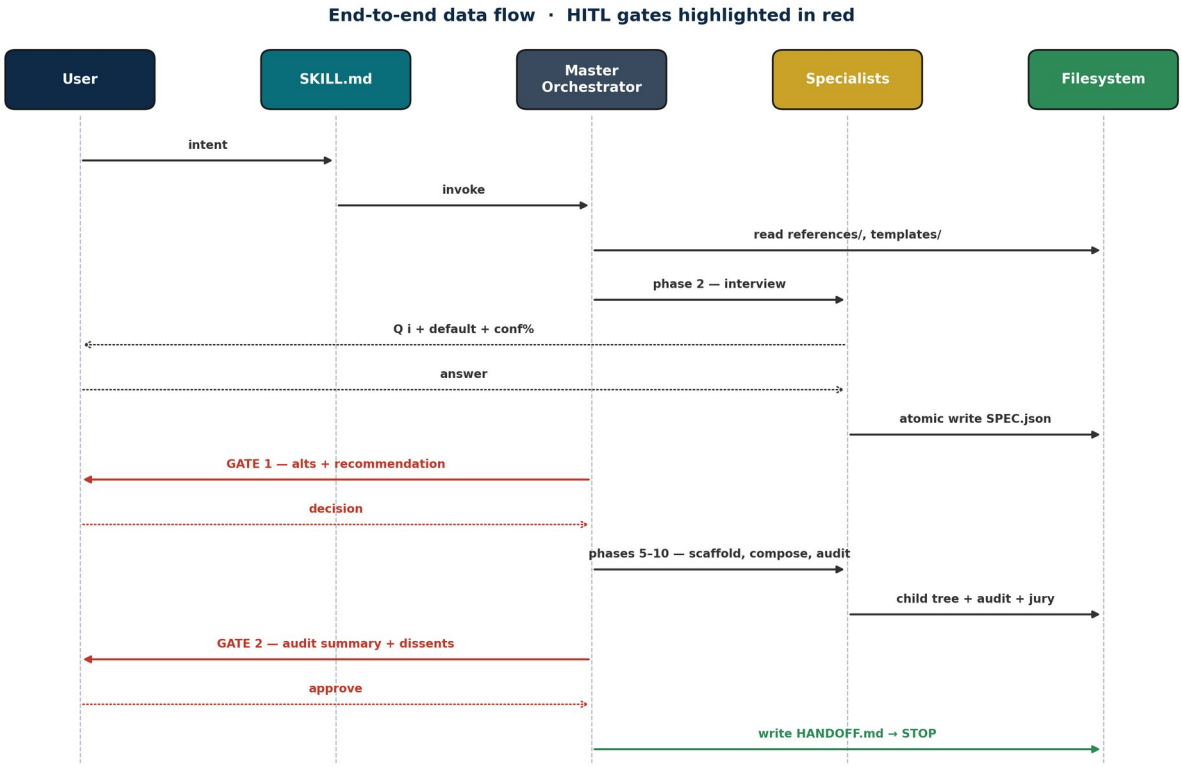

Figure 5 · End-to-end data flow as a sequence diagram. The user appears at the boundaries (intent + HITL gates); the master orchestrator routes everything in between.

5.2 Memory Module

The memory module instantiates a four-typed persistent memory file system in every child project. Memories survive across sessions and across LLM invocations, enabling the child orchestrator to behave coherently over time without re-reading the entire codebase each turn.

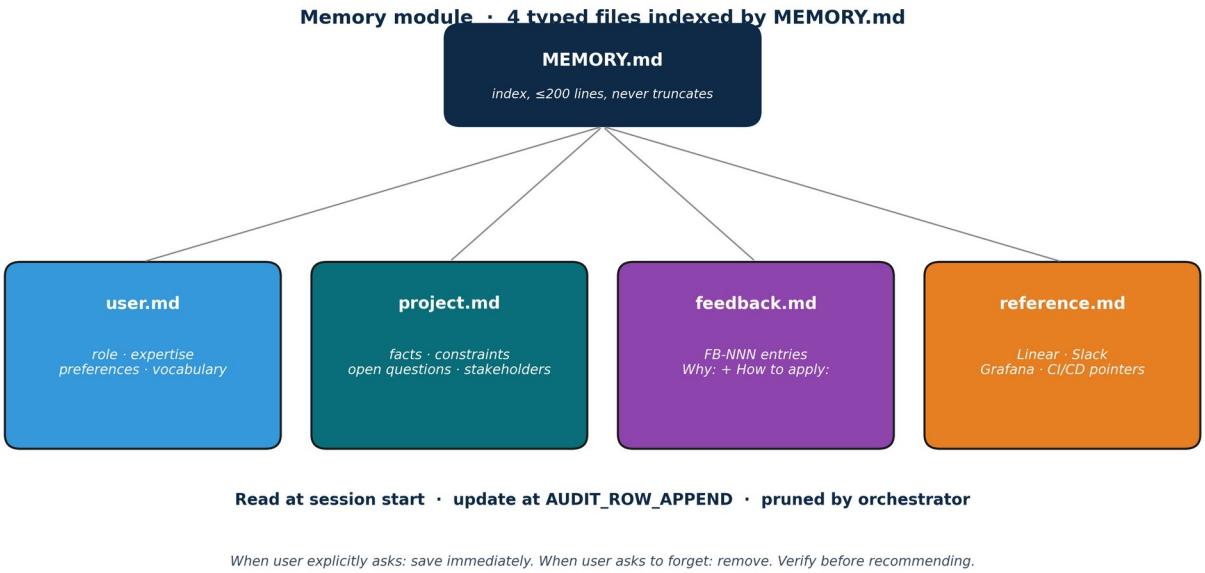

Figure 6 · Memory module structure. MEMORY.md is a one-line-per-entry index; each typed file holds the actual memories.

Four memory types are recognised:

- **user** · user.md — role, preferences, expertise, vocabulary, validated behaviours.
- **project** · project.md — facts, motivations, constraints, decisions taken vs tentative.
- **feedback** · feedback.md — FB-NNN entries with the rule, a Why: line (the reason the user gave) and a How to apply: line (when/where it kicks in). Both corrections and validated successes are captured.
- **reference** · reference.md — pointers to external systems (Linear, Slack, Grafana, CI/CD).

The discipline differs from log-style memory in three ways. First, memories are typed: the four files are semantically distinct, preventing the common failure of conflating user preferences with project facts. Second, every feedback entry carries Why and How-to-apply lines, allowing future-self to judge edge cases instead of blindly following the rule. Third, before recommending from memory, the system verifies the named entity still exists (grep for the function, check the file) — a memory is a claim about state at write time, not a guarantee about the present.

5.3 Task Classification (Interview Wizard)

File: prompts/01_interview_agent.md. Configuration: wizard/interview_questions.json (30 questions across 8 phases) plus wizard/defaults.json (sensible defaults with confidence% and rationale).

The wizard is the only prompt that talks to the user outside HITL gates. It asks one question per turn, validates each answer against a schema (type, options, regex), and persists atomically to SPEC.json (write tmp + rename) before advancing.

The 8 interview phases group questions semantically:

- essentials — system name, version, target path, owner.
- compliance — EU AI Act risk class, additional regulations, domain.
- stack — programming language, frameworks, libraries, model providers.
- methodology — SDD discipline, atomic-writes policy, test-first stance.
- audit — auditor count, auditor mode (parallel/sequential/none), reviewer roster.
- ergonomics — preferred output formats, language for prose, language for code.
- on_demand — which optional artifacts are wanted (dashboard, slash commands, ...).
- spec_seed — initial intent paragraph, top-3 stakeholders, must-have features.

Conservative defaults are enforced in the compliance phase: if domain is in {healthcare, fintech, legal, public_sector} and the user picks risk other than high, the wizard blocks and demands an explicit rationale before advancing. Silent risk-class downgrades are forbidden — they would invalidate the audit baseline and create a hidden compliance gap.

5.4 Per-Session Error Learning

system-designer seeds the child system with a 30-entry catalog of common AI coding errors (AIE-001 through AIE-030) drawn from references/ai_error_catalog.md. The catalog is pre-classified into six categories and supports auto-extension during project life.

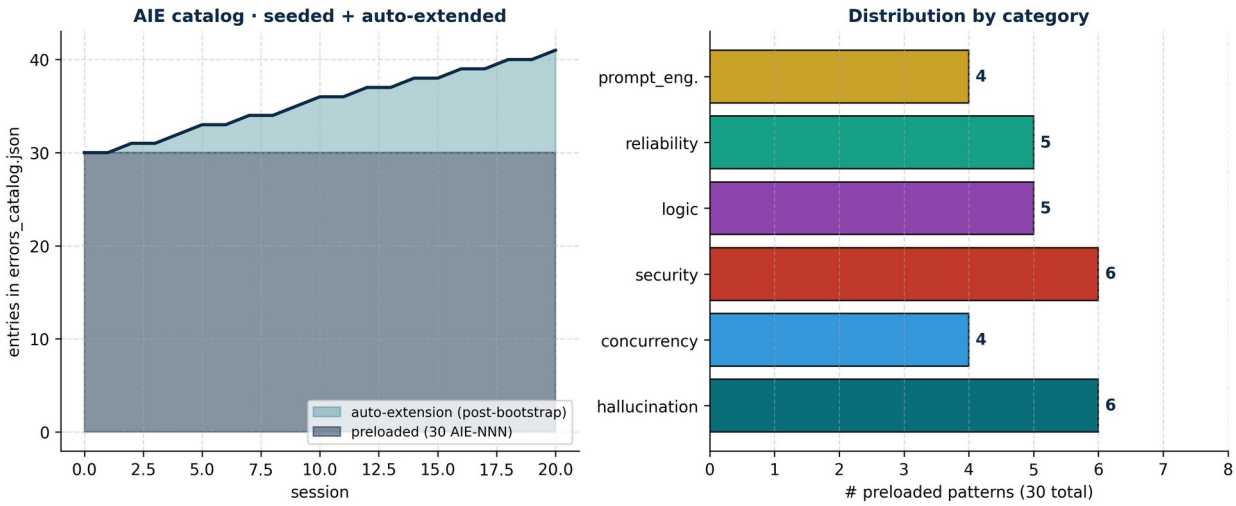

Figure 7 · Error learning per session. Left: the catalog grows beyond the 30-entry preloaded baseline as new patterns are observed. Right: distribution of preloaded patterns by category.

Per-entry schema (each AIE-NNN):

- id — stable identifier (AIE-001 … AIE-030 + auto-extension).
- category — hallucination / concurrency / security / logic / reliability / prompt_engineering.
- severity — low / medium / high / critical.
- description, example, prevention — each calibrated and sourced.
- auto_detection — deterministic (regex on artifact) or heuristic (LLM judge).
- occurrences — incremented every time the pattern is observed in a session.
- first_seen_session — for auto-extended entries (preloaded entries: null).
- preloaded — boolean, distinguishes seeded vs auto-extended entries.

Per-session learning is operationalised via the child orchestrator's AUDIT_ROW_APPEND step. When an error matches an AIE pattern, the child orchestrator: (1) increments occurrences in tracking/errors_catalog.json; (2) appends an ERR-NNN narrative entry to tracking/errors.md with the session ID, the matched pattern, the fix applied, and a calibrated assessment of whether the prevention rule needs reinforcement; (3) if no AIE matches, proposes a new AIE-NNN entry through the auto-extension protocol — gated by HITL at the next session checkpoint to prevent runaway catalog growth.

> *Auto-extension is bounded by HITL approval. The catalog grows under human supervision, not silently. This protects against the failure mode of the system inventing dubious categories to inflate self-perceived coverage.*

5.5 HITL Governance

Two gates are inviolable at scaffold time. Their format is standardised: the user always sees calibrated alternatives with fit-percentages summing to 100%, pros and cons per option, an explicit recommendation with its own confidence percentage, and a free-form branch for off-menu requests.

Beyond scaffolding, the HITL discipline extends into the child system. Every session ends with CHECKPOINT_HITL, where the child orchestrator surfaces lowest-confidence decisions and any auditor dissents. The session does not advance without human acknowledgement.

5.6 Audit Module (3-Auditor + Meta-Jury)

Files: prompts/06_eu_ai_act_mapper.md, prompts/07_audit_designer.md, prompts/09_three_auditors_jury.md. Configuration: SystemSpec.auditors_count (3 to 10) and SystemSpec.auditor_mode (parallel / sequential / none).

EU AI Act 2024/1689 · high-risk mapping · 13 checklists, ≥112 audit rows

Checklist (Spanish AESIA)	EU AI Act Article	min rows (high-risk)
Risk management	Art. 9	12+6
Risk mgmt. example	Art. 9	–
Data governance	Art. 10	14
Technical documentation	Art. 11	18
Record-keeping	Art. 12	8
Transparency to deployer	Art. 13/50	10
Human oversight	Art. 14	8
Accuracy	Art. 15	6
Robustness	Art. 15	6
Cybersecurity	Art. 15	10
Quality management	Art. 17	12
Post-market monitoring	Art. 72	10
Incident reporting	Art. 73	10

Cross-regulations by domain: GDPR · MDR · DORA · ISO 42001 · ISO 13485

Figure 8 · EU AI Act 2024/1689 mapping. 13 AESIA checklists map 1:1 to mandatory high-risk Articles; total floor for high-risk is ≥112 audit rows.

The audit module produces a 14-sheet Excel (00_README + 01..12 per Article + 99_Status_Log) or a CSV+MD fallback when xlsx capability is unavailable. Rows are baselined at scaffold time with status pending; the child orchestrator appends rows session by session — the sheet is never frozen.

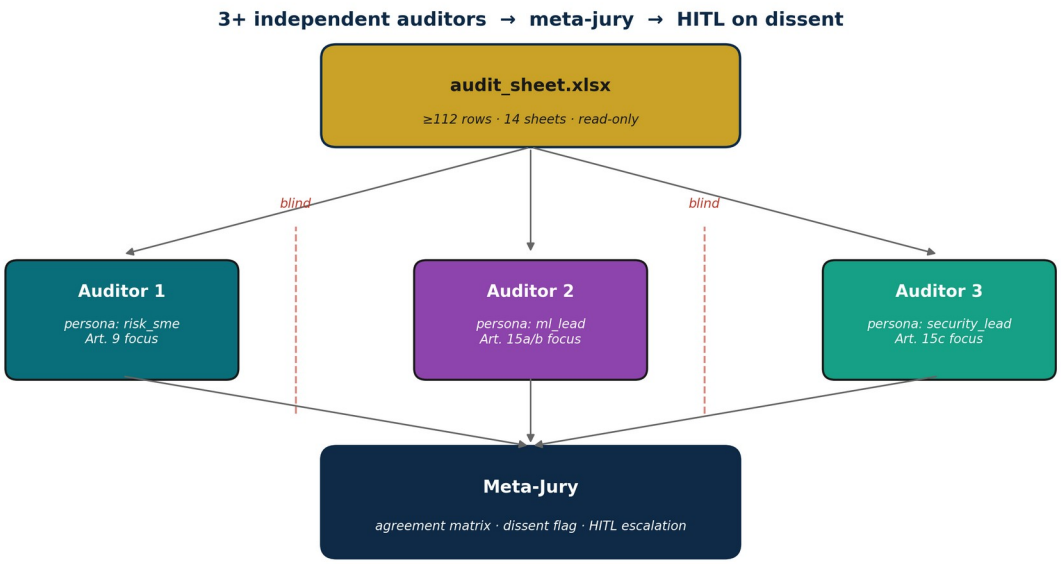

Figure 9 · 3-auditor + meta-jury topology. Auditors are blind to each other (red dashed barriers). Jury aggregates with confidence weighting; dissent always escalates to HITL.

The multi-auditor topology defaults to three auditors with distinct personas (risk-SME, ML-lead, security-lead) for diversity of thought. Auditors run blind to each other — either in parallel (when parallel.spawn capability exists) or sequentially with information barriers. The meta-jury then builds an agreement matrix per row, computes a confidence-weighted consolidated status, and flags two failure modes: dissent (≥2 auditors give conflicting status) and low-confidence consensus (all agree but mean confidence < 70%). Both flags trigger mandatory HITL escalation; dissent is never silently resolved.

5.7 Calibration Engine

Calibration is enforced at three layers. At write time, a regex-based test (T2) rejects the seven forbidden tokens (best, always, never, guaranteed, certain, definitely, impossible). At runtime, every estimate emitted by any prompt carries a confidence% or a range. At audit time, the multi-auditor system requires confidence% on every row status; the jury then aggregates with confidence weighting rather than simple majority.

The four confidence thresholds drive downstream behaviour:

- 90–99% — proceed silently.
- 70–89% — log to session record, proceed.
- 50–69% — surface in the next reflection as a low-confidence decision.
- Below 50% — escalate to HITL at the next gate.

Figure 10 · 11 per-session KPIs with calibration. Estimates carry confidence percentages; measured values do not.

5.8 Library-Doc Fetcher

File: prompts/04_library_docs_fetcher.md. The fetcher uses a four-step fallback ladder to obtain fresh documentation for every library declared in the project's stack: Context7 MCP → primary URL → fallback URL → GitHub release API. Failed network access produces an OFFLINE.md placeholder with a logged warning and reduced confidence; the scaffold does not halt.

Every fetch records source_used, sha256 of fetched content, and last_fetched_at into library_docs/<lib>/<version>/MANIFEST.md. A separate api_index.json is extracted when possible, allowing the child orchestrator to verify any function call against the actual library surface — a direct mitigation of AIE-002 (fabricated dependency).

5.9 Living Documentation

The fifth principle (P5) materialises as a continuous append discipline. tracking/project.json records phase progression, completed phases, gate statuses, emitted artifacts (with sha256), audit results, observed AIE patterns, running KPIs, and calibration / portability violation counters. tracking/sessions/<id>/session.json captures per-session detail. tracking/decisions.md is an append-only ADR log. audit/audit_sheet.xlsx grows session by session. docs/reports/cumulative.md is updated at every milestone.

Atomic writes are the operational invariant. Every output file is written via fs.write(target.tmp) followed by fs.rename(target.tmp, target). A crash between the two operations leaves a recoverable .tmp file but never corrupts the original.

5.10 Reporting Layer

File: prompts/08_report_writer.md. The reporting layer scaffolds the scientific publication infrastructure required for a credible deliverable. Standard selection follows a deterministic tree: healthcare +

predictive system → TRIPOD-AI; healthcare + clinical trial → CONSORT-AI; healthcare + diagnostic study → STARD-AI; healthcare + protocol → SPIRIT-AI; otherwise → IMRaD.

Two universal documents are always emitted regardless of the chosen standard: cumulative.md (the living doc) and imrad.md (the universal fallback). Content placeholders are preserved literally; the report writer never auto-fills content. The child orchestrator fills sections at milestones with calibrated claims, every claim carrying CI / range as the standard requires.

6. Extensions (v0.2.0 · v0.3.x)

Version 0.2.0 inserts four new phases at fixed points in the 13-phase base architecture and adds a cross-phase hook that fires at the end of every task and every session. The additions address three operational gaps observed in pilot v0.1.0 runs: (a) the absence of a calibrated, project-specific knowledge corpus before the interview phase; (b) the lack of a structural-consistency check between scaffolding and Gate #2; and (c) the absence of a lifelong-learning loop that captures human feedback after handoff and audits any system improvement before merge.

Version 0.3.0 then identifies a deeper gap: memory itself was generic (the four-typed Anthropic baseline only) and not contracted per project. Sub-section §6.6 covers the phase 4.5 memory-schema negotiator that fixes this; §6.7 covers the v0.3.1 expansion of the format taxonomy that makes the contract truly fit-for-purpose. All extensions inherit the five core principles (P1–P5) and compose every emitted prompt via prompt-architect.

> *Calibration of this section · ~85% confidence on the design choices (anchored to v0.2.0 self-test suite at 12 PASS / 0 FAIL); ~75% confidence on the long-term fit of the calibrated thresholds (15 corrections / 70% confidence floor / 80% APPROVED ratio) since they are calibrated against a small pilot set and may need adjustment after wider use.*

6.1 Phase 1.5 · Context Curator

Inserted between read_context (phase 1) and interview (phase 2). Builds the project's task-specific knowledge corpus before any planning happens. Sources include implementation guides, legislation when applicable, non-library documentation, peer-reviewed papers, expert blogs, forum threads, and curated internet opinions. Each source carries a calibrated confidence percentage drawn from a 10-category taxonomy (legislation_official 95–99% · regulator_implementation_guide 88–95% · standards_body_doc 85–92% · vendor_official_docs 85–92% · peer_reviewed_paper 80–90% · industry_standard_doc 80–90% · expert_blog 60–75% · forum_discussion 30–50% · internet_opinion_general 20–50% · user_uploaded_doc 80% overridable). Library documentation is explicitly out of scope here — that remains phase 7's domain.

The fetch tool ladder tries mcp.playwright (recommended) first, then plain fetch(), then emits a download_recommendations.md row asking the user to upload sources the AI cannot reach (paywalled, login-gated, or locally stored). The mandatory HITL fetch-plan approval before any network call protects against accidental over-collection. The corpus is living: the child orchestrator inherits the curator and re-runs it at every session boundary in living_update mode. Pruning is deterministic and bounded — only sources with confidence ≤50% AND a tested-unhelpful flag from the session log are eligible. Sources are never pruned solely on age.

- **Outputs** · ≈30 invariants enforced via per-source manifest.json and the consult_websites.md / download_recommendations.md split.
- **Conservativeness** · Internet opinions default to consult_websites_only=true unless the user opts in (protects against drift).

- **Provenance** · Failed fetches are recorded with the full fallback chain in the manifest, never silently dropped.

6.2 Phase 11.5 · Data Flow Validator + Simulation Agent

Inserted between reflection (phase 11) and Gate #2 (phase 12). Captures a structural snapshot of the freshly-scaffolded child tree and dispatches n ∈ [3, 10] specialist validators plus one mandatory simulation agent. The validator count is task-driven by a calibrated formula:

> *n_validators = clamp(3, ceil(artifacts/30) + ceil((100−audit%)/20) + (eu_risk==high ? 2 : 0), 10)*

Validator personas are drawn from a menu of ten specialised lenses: data_lineage_auditor, memory_consistency_auditor, intercom_auditor, schema_integrity_auditor, lifecycle_auditor, calibration_consistency_auditor, portability_consistency_auditor, error_catalog_drift_auditor, and eu_act_traceability_auditor. The first three are the floor; the rest are added by the formula based on importance signals. Every validator is composed fresh via prompt-architect (P4) and runs in parallel when the runtime supports it; sequential fallback is documented in the consolidated report.

The simulation agent is non-optional (P3 does not apply to phase 11.5). It runs five synthetic scenarios as dry-run traces against the just-scaffolded child tree: (S1) resumability — verify that an interrupt at phase 5 leads to unambiguous resumption from tracking/project.json#current_phase; (S2) library-doc fetch failure — walk the fallback ladder and confirm it ends at OFFLINE.md without aborting; (S3) jury dissent preservation — inject conflicting auditor verdicts and confirm dissent surfaces (not averaged); (S4) calibration drift — inject an 'always' token and confirm the regex + LLM scan both fire; (S5) atomic-write race — sample five writes from observations.jsonl and confirm each shows the tmp_path → final_path pattern.

The consistency score is derived deterministically:

> *score = 100 − 5×dissents − 3×low_confidence_findings − 2×partial_status − 10×any_failed_simulation*
> *clamp [0, 100]*
> *score < 80 → Gate #2 escalation (no auto-pass)*

6.3 Phase 13.5 · Feedback Learning Loop

Fires at the very end of every fully-audited session, immediately after handoff. The loop captures human feedback as free text, classifies each correction under a calibrated taxonomy (severity × category × recurrence + confidence percentage), and persists to a hybrid SQLite + Markdown store. The DB is authoritative; the MD mirror is regenerated from the DB at the end of every run, so the two never drift.

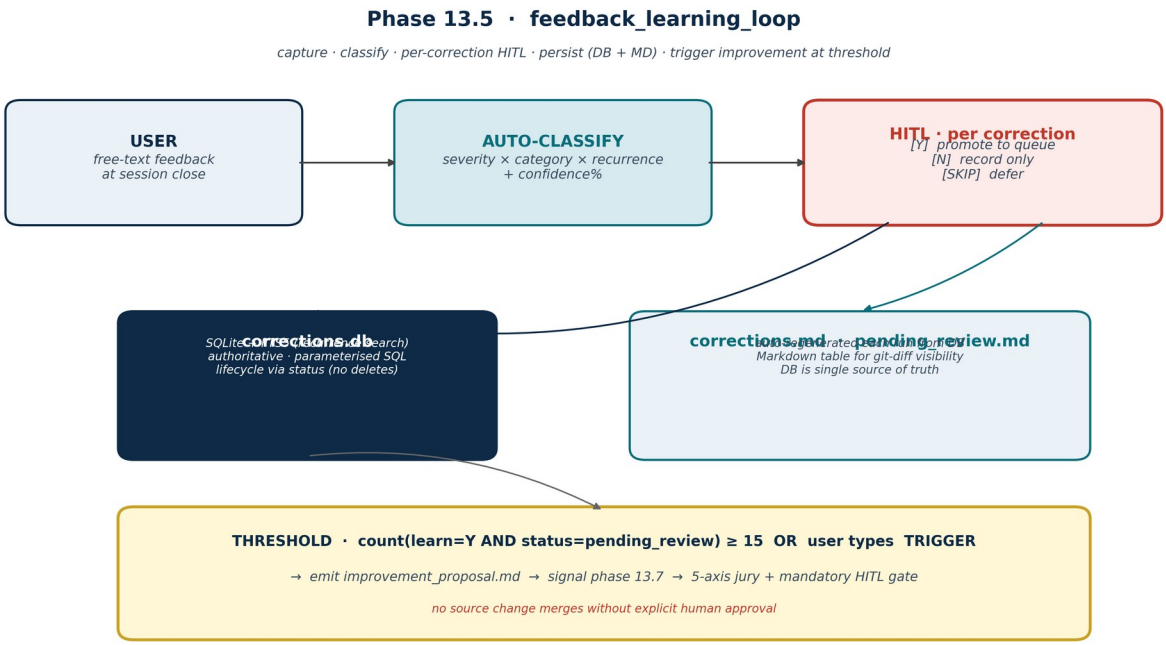

Figure 12 · Feedback learning loop. User feedback flows through auto-classification (severity × category × recurrence with confidence%), then a per-correction HITL prompt asks 'should the system learn from this? [Y / N / SKIP]'. Persistence is dual: SQLite + FTS5 for indexed search and threshold counting; Markdown mirror for git-diff visibility. When the pending learn-Y counter reaches the threshold (default 15) or the user types TRIGGER, an improvement_proposal.md is emitted and phase 13.7 is signalled. No source change merges without explicit human approval at phase 13.7.

The per-correction HITL block is the design decision that protects the system against noise pollution. After every correction is classified, the user sees a one-screen summary and answers Y / N / SKIP. The system never defaults this field — every value reflects an explicit human keystroke. Y promotes the row to the improvement queue; N records it for traceability only; SKIP defers it to the next session. The threshold rule then becomes deterministic: when the count of (learn_in_system=Y AND status=pending_review) reaches learn_threshold (default 15) — or when the user types TRIGGER — the system emits feedback_learning/improvement_proposal.md and signals phase 13.7.

The taxonomy itself is calibrated. Severity is one of {info, warn, error, critical}. Category is one of nine buckets (calibration, portability, memory, prompt_architect, HITL, EU_AI_Act, tooling, documentation, other). Recurrence is one of {one_off, recurring (FTS5 hits ≥2), systemic (≥4 hits OR cross-category cluster)}. A recurring hit on the same feedback signature across sessions automatically promotes the row's recurrence field, feeding the importance score that downstream improvements consider.

6.4 Phase 13.7 · Improvement Jury (5 Specialist Axes)

When phase 13.5 emits an improvement proposal, phase 13.7 audits it along five fixed axes in parallel. The fixed-five design is intentional: these are the axes that protect the system's invariants regardless of what the proposal touches.

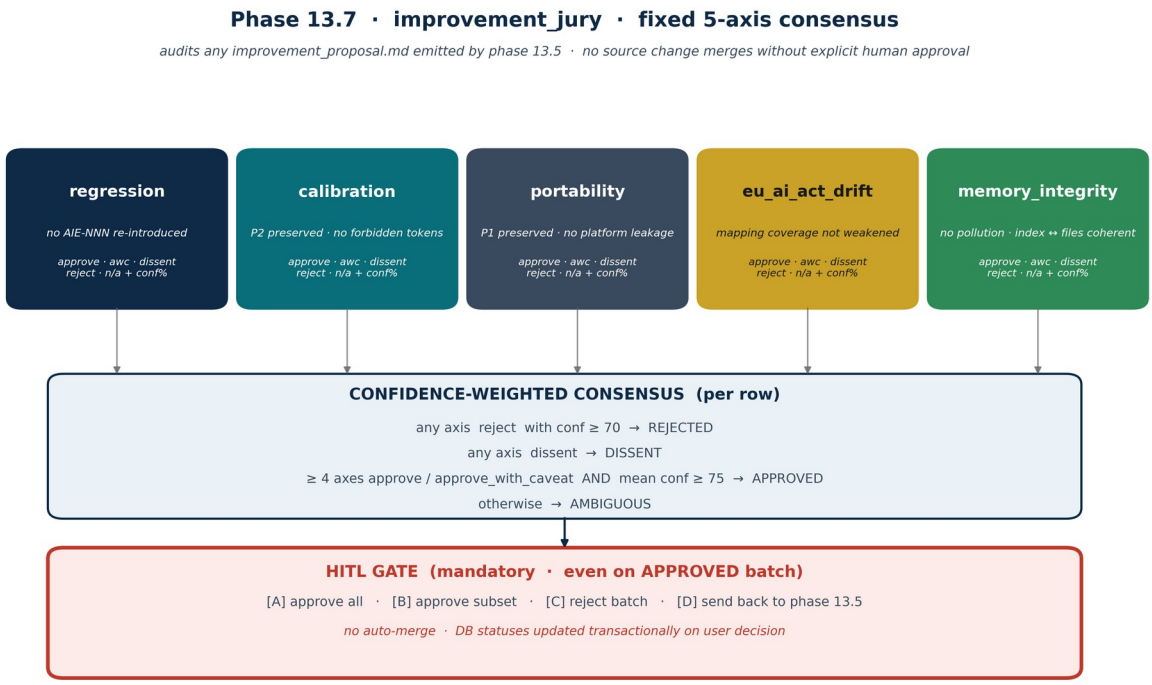

Figure 13 · Improvement jury topology. Five specialist auditors run in parallel (regression, calibration, portability, eu_ai_act_drift, memory_integrity). Each emits a verdict with confidence%. The consensus rule is confidence-weighted but dissent-respecting: any reject ≥70% blocks; any dissent surfaces; ≥4 approve with mean conf ≥75% approves; everything else is AMBIGUOUS. The HITL gate at the end is mandatory regardless of batch verdict — there is no auto-merge.

Axis	What it protects
regression	no AIE-NNN re-introduced; no phase-11.5 invariant broken; no prior consolidated row reverted to dissent
calibration	P2 preserved; no forbidden tokens introduced; existing % bands not weakened
portability	P1 preserved; no Claude-Code-only or platform-specific identifiers added; abstract tool descriptions retained
eu_ai_act_drift	audit/eu_ai_act_mapping.md coverage not weakened; checklist links intact; risk class not lowered without decisions.md rationale
memory_integrity	no entries added without explicit learn_in_system=Y; index ↔ files coherent; 4-type taxonomy respected

The post-consensus HITL gate is the operational point where the system stops being automated. The user sees a per-row verdict matrix (5 columns × proposal rows) and chooses [A] approve all APPROVED rows; [B] approve a subset (the user lists row_ids); [C] reject the entire batch; [D] send back to phase 13.5 with notes. On approval, corrections.status='approved' is set in a single transaction. On rejection, the proposal is archived and the rows return to status='rejected'. On send-back, statuses remain pending_review and the proposal is marked for redraft.

6.5 Cross-phase · Adaptive Audit Meta-Validator

The adaptive_audit_meta prompt is the general engine that prompts 6.2 and 6.4 specialise. It fires at the end of every task and every session — in both the generator (per-phase) and the inherited child orchestrator. Where the improvement jury at §6.4 uses a fixed five-axis panel for one specific kind of audit, the meta-validator computes how many auditors are needed (3–10) from a calibrated importance score and freshly composes each auditor via prompt-architect with a persona that is an actual expert in the thing being audited.

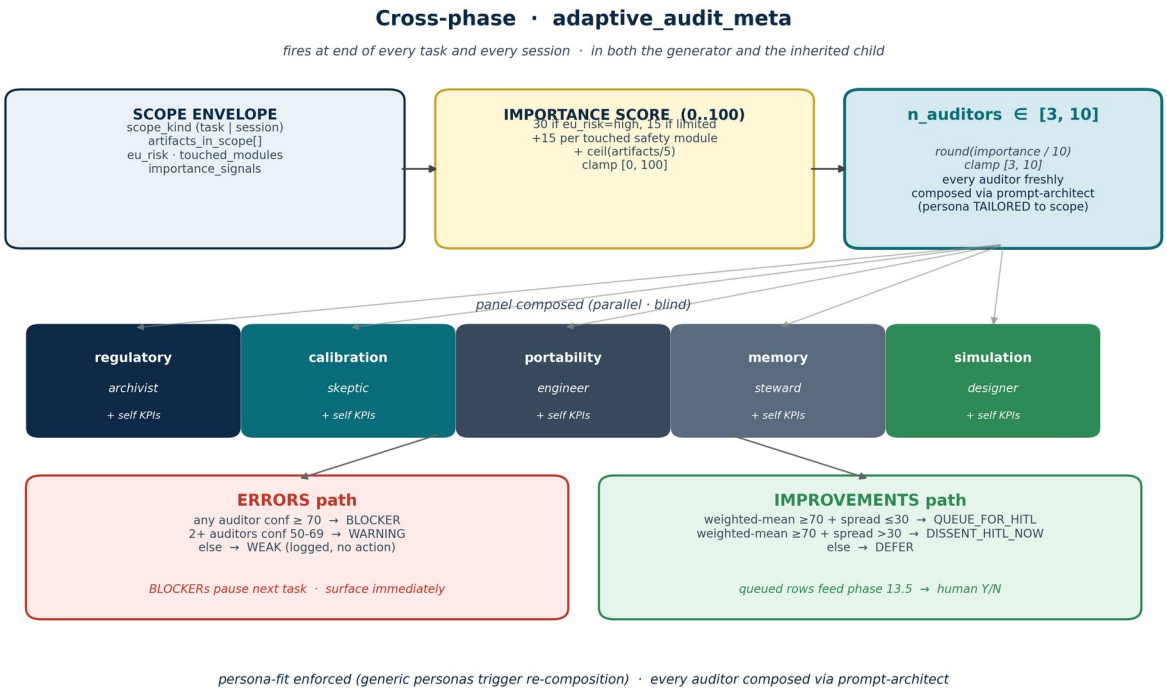

Figure 14 · Adaptive audit meta-validator. The orchestrator passes a scope envelope (task or session). The meta-validator computes an importance score (0–100) from eu_risk × touched_modules × artifacts_in_scope, then derives n_auditors via round(importance/10) clamped to [3, 10]. Each auditor is freshly composed via prompt-architect with a persona TAILORED to the scope and its own KPI block. The panel runs in parallel and produces findings on two separate paths: errors (must address) and improvements (queued for phase 13.5).

The importance score is calibrated:

```
importance =  (eu_risk == high ? 30 : limited ? 15 : 5)
          + (touches safety_floor ? 20 : 0)
          + (touches calibration ? 15 : 0)
          + (touches memory ? 15 : 0)
          + (touches prompt_architect_floor ? 15 : 0)
          + (touches HITL_logic ? 15 : 0)
          + (touches eu_ai_act_mapping ? 10 : 0)
          + ceil(artifacts_in_scope / 5)
          clamp [0, 100]
n_auditors = clamp(3, round(importance / 10), 10)
```

Examples calibrated against the v0.2.0 pilot: renaming a documentation heading produces importance ≈ 5 → n=3; updating a calibration scan produces importance ≈ 50 → n=5; editing the master orchestrator's HITL logic in a high-risk healthcare project produces importance ≈ 80 → n=8; scaffolding a fresh high-risk healthcare child produces importance ≈ 95 → n=10.

The persona library is illustrative rather than exhaustive — Factory may compose any plausible expert whose lens is necessary for the scope. The reflection step explicitly checks for generic personas (slugs containing 'generic_*') and triggers re-composition (maximum 3 retries). This is what keeps the panel adaptive in practice rather than just in name.

Findings split into two consensus paths. On the error path, any auditor with confidence ≥70% on a finding makes that finding a BLOCKER — the orchestrator pauses the next task until the blocker is addressed. Two auditors with confidence 50–69% on the same group make it a WARNING that surfaces at the next HITL gate. Anything weaker is logged but not actioned. On the improvement path, the consensus rule blends weighted-mean confidence with spread: weighted-mean ≥70% with spread ≤30 percentage points queues the row for HITL review at phase 13.5; weighted-mean ≥70% with spread >30 surfaces immediately as DISSENT_HITL_NOW; everything else defers.

> *Why dynamic n_auditors? The historical fixed-N model treated every audit the same. v0.2.0 makes audit depth proportional to risk: the structural-consistency formula loads on artifacts × audit gap × eu_risk; the adaptive-meta formula loads on eu_risk × touched modules × artifacts in scope. Both clamp at [3, 10] — the lower bound protects against under-audit on small scopes; the upper bound caps cost. The improvement-jury at 13.7 remains intentionally fixed at 5 (the five axes that protect the system's invariants).*

Together, the four new phases plus the adaptive cross-phase hook form what the meta-skill documents call 'a meta-generator of validation-layer auditors inside the meta-generator of AI architectures'. The intent is that audit depth, audit composition, and audit discipline all become first-class outputs of the system rather than fixed conventions baked at design time.

6.6 Phase 4.5 · Memory Schema Negotiator (v0.3.0)

Inserted between Gate #1 (phase 4) and scaffold (phase 5). Negotiates with the human what exactly the memory of THIS project must store — per module, per format, per trigger, with calibrated audit completeness rules. Memory is the foundation: an over-broad schema produces noise; an under-broad schema produces silent failure. This phase makes the trade-off explicit and human-owned.

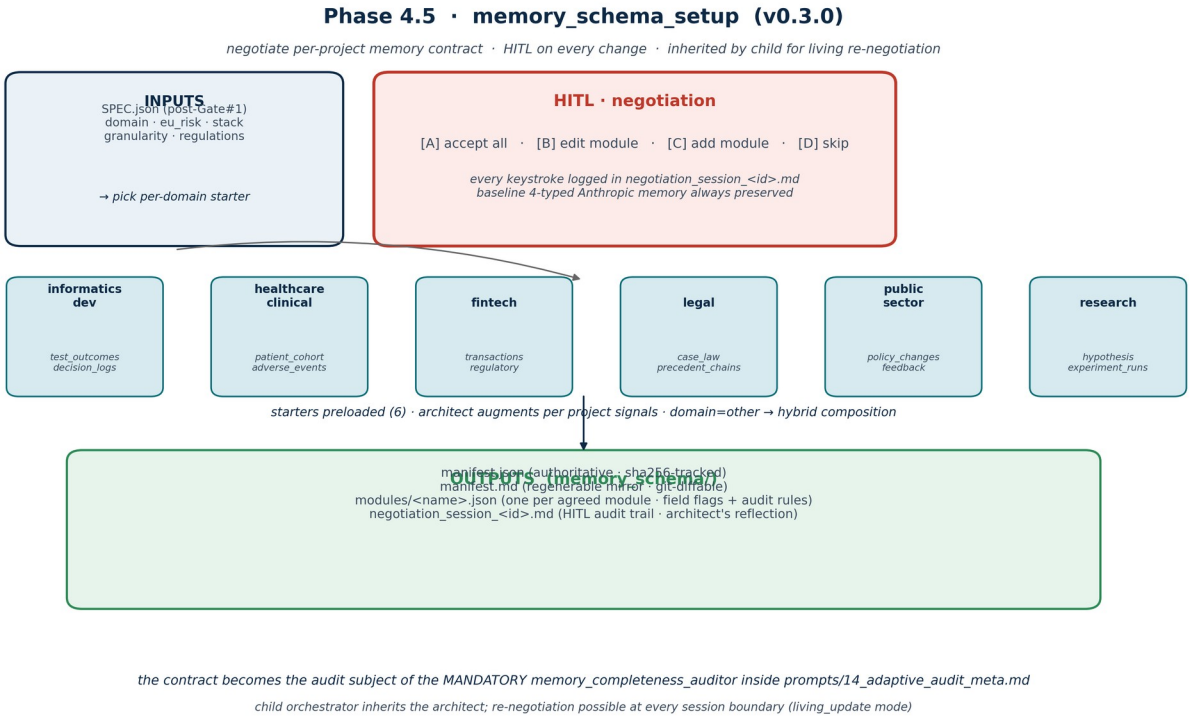

Figure 15 · Phase 4.5 memory schema negotiation. The architect reads the post-Gate-#1 SPEC.json, picks a per-domain starter from the 6-starter library (informatics_dev, healthcare_clinical, fintech, legal, public_sector, research), augments with project-specific signals, and surfaces a HITL block ([A] accept all / [B] edit module / [C] add module / [D] skip). Outputs are persisted to memory_schema/ with atomic manifest.json + regenerable manifest.md mirror + per-module schema files + a HITL negotiation record. The Anthropic four-typed baseline is always preserved alongside the negotiated structured modules.

The contract is encoded with a four-level field-flag taxonomy that captures conditional mandates explicitly. The user's anchor example for software-development projects — "every test #N attempt #N status; if not pass, error_code + suggested_solution + confidence%" — maps directly into the informatics_dev starter's test_outcomes module:

field	flag	rationale
test_number, attempt_number, status	mandatory	always present (test_id, ordering, success?)
error_code, error_message	mandatory_if_status!=pass	structured handle + human-readable, only when there's a failure
suggested_solution	mandatory_if_status!=pass	actionable fix proposal
suggested_solution_conf_pct	mandatory_if_suggested_solution	P2 calibration on the proposal
related_correction_id	recommended	FK into feedback_learning/corrections.db when known
session_id, ts, agent_confidence_pct	mandatory	lineage + temporal context + P2

The mandatory memory_completeness_auditor persona inside prompts/14_adaptive_audit_meta.md (promoted from optional in v0.2.0 to mandatory in v0.3.0, analogous to the simulation_agent in phase 11.5) reads this contract on every task and every session and runs a two-tier audit:

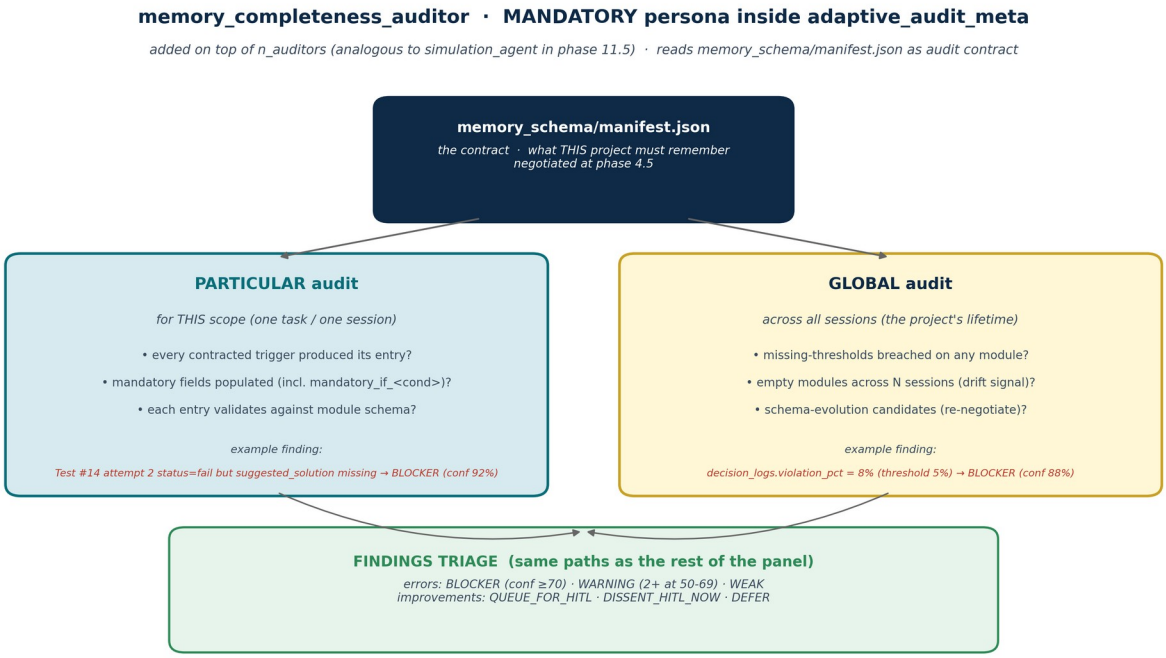

Figure 16 · Two-tier memory completeness audit. The mandatory auditor reads memory_schema/manifest.json as its contract and runs both tiers on every adaptive_audit invocation. Particular tier (left): for the scope at hand, did each contracted trigger produce its entry? Are mandatory fields populated? Global tier (right): across all sessions, are the missing-thresholds breached? Are any modules empty across N sessions? Both tiers feed the existing finding-triage paths (BLOCKER / WARNING / WEAK + QUEUE_FOR_HITL / DISSENT_HITL_NOW / DEFER).

Threshold defaults are calibrated: 5% missing-rate triggers a BLOCKER on the global tier for medium-stakes modules; 1% for high-risk regulated modules (e.g., adverse_events in healthcare). High-risk regulated modules carry a non_removable_for_high_risk flag in the starter; attempts to remove them at HITL surface a regulatory warning logged to decisions.md (overridable, but visibly so).

6.7 8-Format Memory Taxonomy + Selection (v0.3.1)

Version 0.3.1 expands the format taxonomy from three options (json / jsonl / structured_md) to eight, organised in two portability tiers. The architect ALWAYS picks the best format per module — a blanket choice across all modules is a silent-failure mode the protocol explicitly refuses.

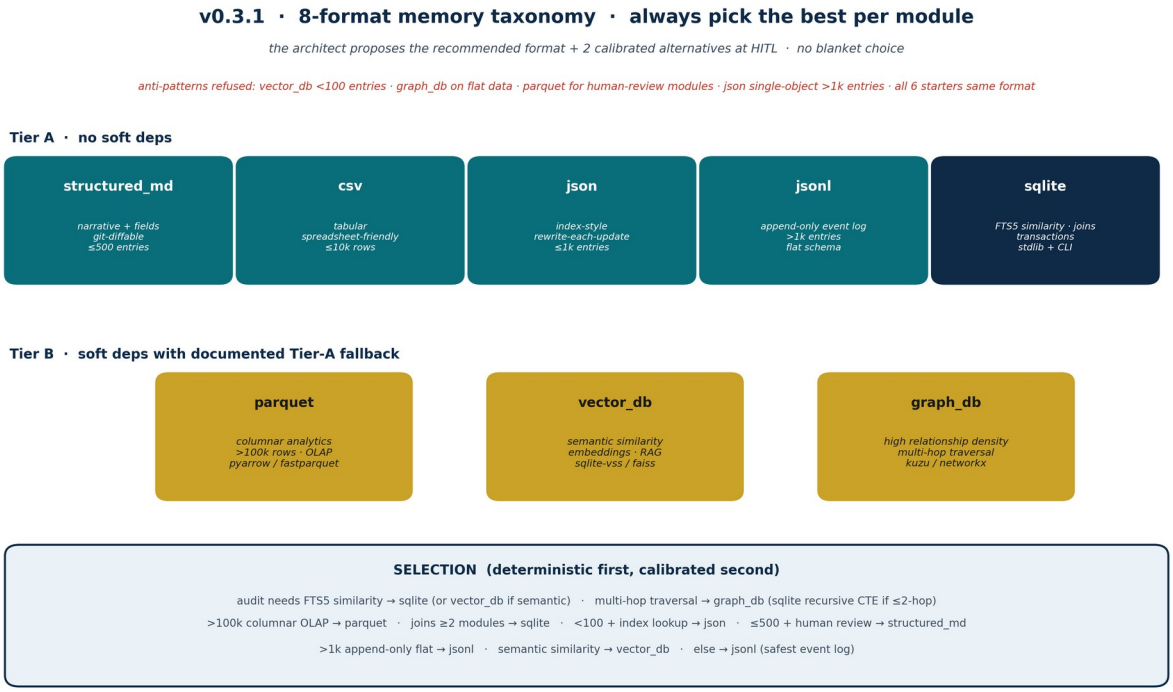

Figure 17 · 8-format memory taxonomy with portability tiers and selection rules. Tier A (top row, no soft deps): structured_md, csv, json, jsonl, sqlite. Tier B (middle row, soft deps with documented Tier-A fallback): parquet (columnar analytics), vector_db (semantic similarity), graph_db (multi-hop traversal). Bottom band: the calibrated selection rules — deterministic first (audit-rule needs FTS5 → sqlite; multi-hop traversal → graph_db; etc.) and calibrated second (volume + query pattern thresholds). Each module surfaces the chosen format + 2 alternatives with fit% at HITL.

Selection rules are deterministic-first, calibrated-second. The architect refuses anti-patterns at HITL (vector_db for fewer than 100 entries, graph_db on flat data, parquet for human-review-primary modules, json single-object for more than 1k entries, all six starter modules picking the same format). The audit-rule × format compatibility matrix in references/memory_schema_protocol.md prevents another silent-failure mode: an audit rule that needs FTS5 similarity dedupe cannot run natively on jsonl, so the architect must either upgrade the format to sqlite or declare an offline FTS sidecar fallback.

Examples calibrated against the v0.3.1 starter upgrades: legal/precedent_chains moved from jsonl to graph_db (multi-hop precedent traversal is the canonical case); fintech/transaction_pattern_audits moved to sqlite (FTS5 over pattern_signature enables cross-session dedupe); healthcare/patient_cohort_signatures moved to sqlite (joins with model_calibration_per_subgroup and trial_arm_assignments). The user's anchor example (informatics_dev/test_outcomes) keeps jsonl as its primary choice because append-only high-volume flat-schema event logging is exactly what jsonl is best at, but the manifest now carries sqlite (fit 80%) and parquet (fit 25%) as documented alternatives for future re-negotiation when volume passes ~5k entries.

> *Why the taxonomy expansion matters · A single format cannot serve every workload. Without FTS5 (sqlite), pattern-similarity dedupe becomes O(N) regex on jsonl. Without graph traversal, legal precedent chains become string lookups. Without columnar analytics, large research hyperparameter sweeps require full-file rewrites. The 8-format taxonomy plus the calibrated selection matrix plus the HITL-surfaced alternatives make the right choice the default, and document the trade-off when a Tier-B format*

> *is unavailable. Every Tier-B format carries a documented fallback to a Tier-A format, so no project becomes unrunnable when a soft dep is absent.*

6.8 External-Audit Closure (v0.3.2 → v0.4.0 → v1.0.0)

After the v0.3.1 release, the system was submitted to a blind 3-axis external-audit panel. Three specialist auditors ran in parallel, each in an independent Opus 1M context window, with the explicit instruction to stay in their lane and avoid cross-axis encroachment. A consensus jury then synthesised the three reports applying the project's own jury_consensus_protocol.

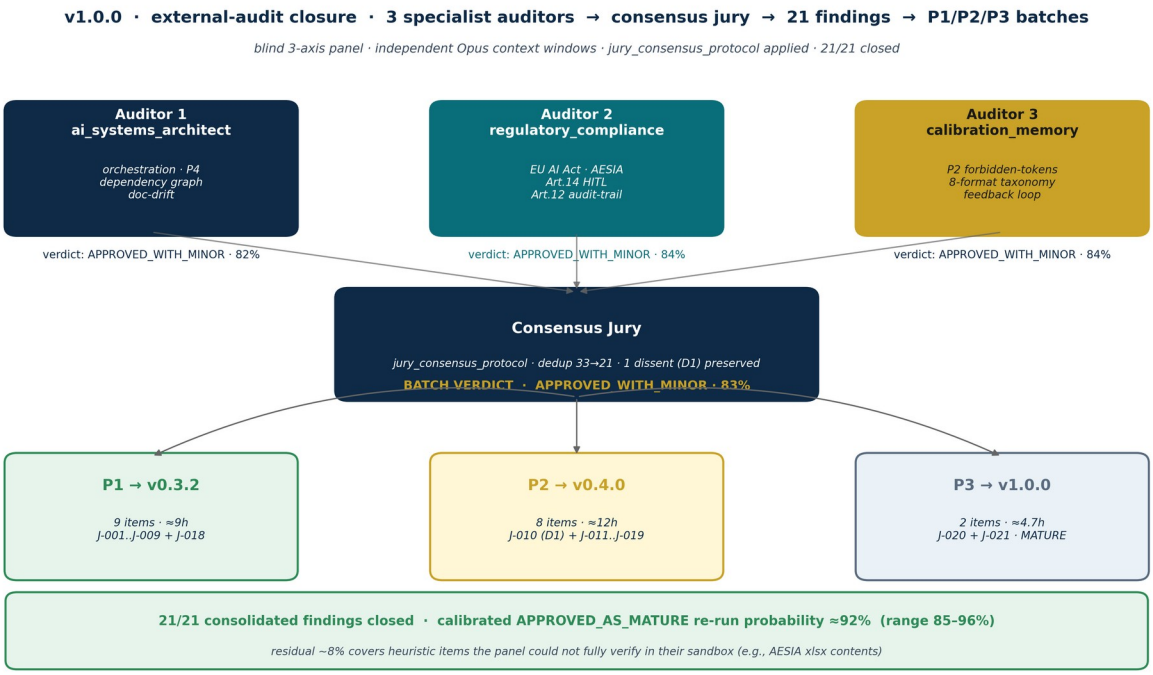

Figure 18 · External-audit topology and 21-finding closure pipeline. Top row: the three specialist auditors (systems-architect / regulatory-compliance / calibration-memory-feedback) ran blind in independent Opus context windows; each delivered a structured report with calibrated findings. Middle row: the consensus jury de-duplicated 33 raw findings into 21 consolidated items, preserving one cross-axis disagreement (D1, on audit-trail-integrity nuance), and reached APPROVED_WITH_MINOR at 83% confidence. Bottom row: the three release batches that shipped the full remediation roadmap. Bottom strip: the closure status — 21/21 items closed; calibrated re-audit probability of APPROVED_AS_MATURE ≈92% (range 85–96%).

The remediation roadmap shipped in three releases. Each batch was self-audited via the system's own test suite (which the audit itself improved as part of the remediation).

Release	Batch	Items closed	Effort	Headlines
v0.3.2	P1	9 items + 1 free typo (J-001..J-009 + J-018)	~9 h	non_removable_for_high_risk symmetry across starters · Art.50 unbundled · SystemSpec.role enum · Art.73 deadline workflow · 2 invented XML tags

				canonicalised · T22 hardened against taxonomy · T2 split into T2a heuristic + T2b strict-blocking · dashboard refresh to v0.3.2 · doc-drift fix
v0.4.0	P2	8 items (J-010..J-019)	~12 h	sha256 tamper-evident hash-chain (resolves D1) · Art.72 min_rows 6→10 · special-category-data wizard question · SystemSpec.hitl_mode enum · gate_status modernised · public_sector format diversity · particular-tier worked example · baseline-memory FS-existence check · new T23 deterministic test
v1.0.0	P3	2 items (J-020 + J-021)	~4.7 h	Art.15(4) adversarial-robustness rows (+4) · Art.9(2) iterative-review cadence rows (+3) · MiCA + PSD3 explicit cross-walk for fintech · prompt 14 dependencies declare memory_schema_protocol + manifest as Hard · cache_hint rubric scoped to Complex tier only · wizard placeholders marked is_template

Disagreement D1 deserves dedicated discussion because it was the one finding that survived consolidation as a preserved cross-axis dissent. The systems-architect auditor cited the per-artifact sha256 logging in tracking/project.json#artifacts_emitted as a 'chain' (S5 strength, confidence 93%). The regulatory-compliance auditor pointed out, correctly, that Article 12 audit-trail integrity requires a Merkle-style chain — a chain of hashes where each entry references the prior entry's hash — not just per-entry hashing. The jury preserved both views and resolved them as J-010 (P2). v0.4.0 implements the chain via the prior_hash field and a deterministic test (T23) that verifies tamper detection.

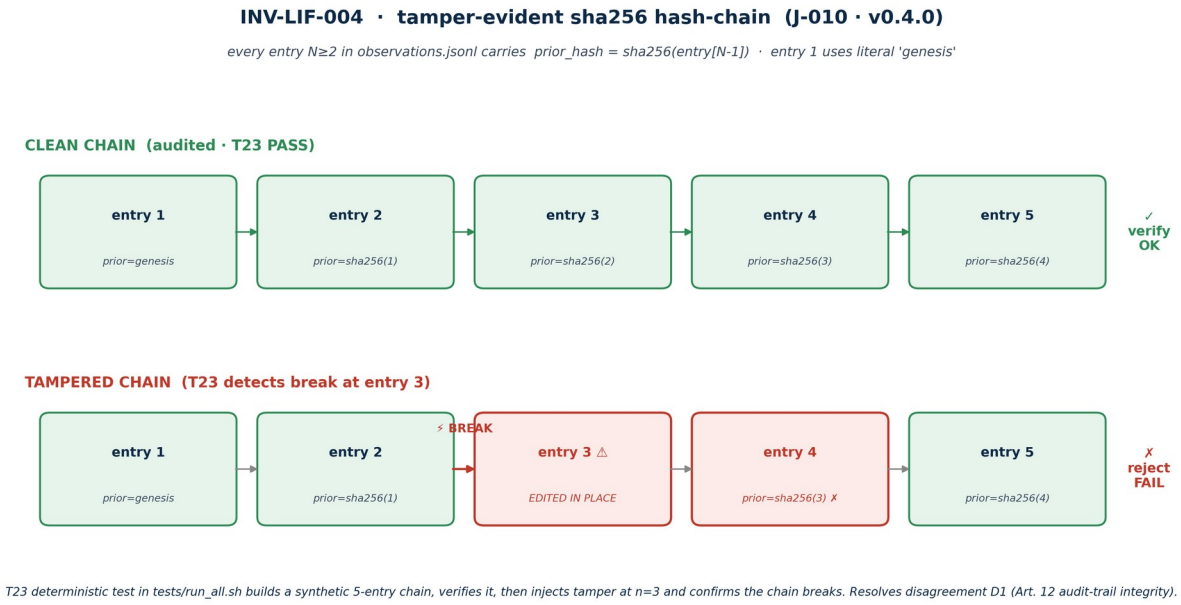

Figure 19 · INV-LIF-004 · tamper-evident sha256 hash-chain shipped in v0.4.0 (J-010). Top row: a clean 5-entry chain; each entry N≥2 carries prior_hash = sha256(serialised entry[N-1]); entry 1 uses the literal string 'genesis'. The chain verifies cleanly (T23 PASS · green check). Bottom row: an in-place edit to entry 3 leaves the recorded prior_hash on entry 4 unchanged, so the chain breaks at the 2→3 transition; the auditor rejects the chain (T23 detects · red ✗). Same discipline applies to tracking/decisions.md ADR rows. The chain is opt-in at runtime: child orchestrators that haven't been regenerated continue to write entries without prior_hash; the audit reports a P2 warn on mixed mode. Resolves the panel's only preserved disagreement (D1).

Total minimum audit rows for high-risk projects increased across the closure window: 112 (v0.3.x) → 116 (v0.4.0, after Art.72 four-sub-system enumeration) → 123 (v1.0.0, after the Art.15(4) adversarial-robustness and Art.9(2) iterative-review additions). The dashboard now surfaces 9 new v0.3.x KPI tiles mapped to tracking/project.json paths (consistency score, memory-schema modules count and contract confidence, feedback pending count and oldest-pending-age, mean classification confidence, adaptive-audit blockers and improvements-queued, and Art.73 open-incident count).

> *Honest accounting · The calibrated 92% probability of APPROVED_AS_MATURE on a re-run is not certainty. The residual ~8% covers heuristic items the panel could not fully verify in their sandbox: the actual contents of the 13 AESIA xlsx checklists (Auditor 2 had Bash python3 invocation denied), prior_hash adoption rate across child projects regenerated from older manifests, and second-order interactions between SystemSpec.role and domain-specific starters that have not yet seen production traffic. v1.0.0 declares maturity, not perfection.*

7. Conclusions

The system-designer meta-generator demonstrates that LLM-driven coding can be made compliant, reliable, and reproducible without sacrificing the productivity gains that make these tools attractive. The design choices that matter most appear to be:

- **Calibration** · Calibration as a first-class output discipline.

By rejecting forbidden tokens and requiring confidence percentages on every estimate, the generator forces uncertainty to be visible. This is the difference between a tool that produces confident-but-wrong outputs and one that produces actionable uncertainty for human review.

- **HITL inviolability** · Inviolable HITL gates, not best-effort suggestions.

By design, no flag, environment variable, or clever LLM behaviour can skip the two scaffold-time gates. EU AI Act Article 14's human-oversight requirement is operationalised at the moment scope and quality are most malleable.

- **Self-application of prompt-architect** · A meta-skill that audits its own meta-skill.

The master orchestrator is itself audited via the prompt-architect skill at level G9=A. This eliminates the common pattern of "do as I say, not as I do" in prompt-engineering frameworks: the framework's own conductor must clear the same bar it imposes on its outputs.

- **Living docs** · Living documentation with append-only invariants.

By making tracking, audit, and reports continuous append targets — and by treating atomic writes as a hard rule — the generator produces deliverables whose integrity is checkable session by session, not only at hand-off.

- **Portability** · Portability as a contract, not an aspiration.

Abstract tool names, fallback ladders for every optional capability, and a regex test that rejects proprietary tool references make the generator usable across at least six Tier-A LLM platforms. This is the operational difference between a meta-skill and a platform-specific automation.

> *These five design choices are mutually reinforcing. Removing any one degrades the others: calibration without HITL produces honest uncertainty no one acts on; portability without auditing produces consistent mediocrity across platforms; living docs without atomic writes produces visible-but-corrupted history. The set is irreducible.*

8. Strengths and Limitations

8.1 Strengths

- **Compliance-by-construction** · EU AI Act 2024/1689 alignment is built-in, not bolted on.

The 13 AESIA checklists are referenced 1:1 to mandatory Articles for high-risk systems. The audit baseline (≥112 rows) is enforced at scaffold time, not deferred to a pre-submission scramble. Risk-class downgrades in high-risk-presumed domains require explicit rationale logged in decisions.md.

- **Calibration** · Calibration discipline is automated.

Forbidden tokens are rejected by a regex test. Every Complex prompt's alignment_rules block cites the calibrated_probabilities reference. The set of seven forbidden tokens is deliberately small and unambiguous — false positives are rare.

- **Prompt quality** · Every emitted prompt clears a 14-rubric audit.

Bypassing the prompt-architect is structurally impossible: only the prompt-factory (prompts/03_prompt_factory.md) writes to the child's prompts/ directory, and the factory always invokes the architect.

- **HITL UX** · Both gates render a standardised, calibrated alternative format.

Users see fit% per option, recommendation with confidence%, and a free-form branch. This matches the cognitive shape of an actual decision rather than a yes/no rubber stamp.

- **Portability** · Six Tier-A platforms supported on day one.

Drag-folder + Read SKILL.md is a universal invocation. The slash-command convenience layer is a thin add-on for Claude Code; the meta-skill works identically on Cursor, Cline, Gemini CLI, Copilot CLI, and Codex.

8.2 Limitations

- **LLM-judged tests are not deterministic** · LLM-judged tests carry residual subjectivity.

Of the 10 tests in the self-validation suite, 6 are deterministic (regex / structural) and 4 are LLM-judged (T3 schema, T4 prompt-tag floor, T5 manifest URLs, T8 atomic-write pattern). The LLM-judged tests have ~85% confidence per execution, which is high but not deterministic. A more rigorous future version would replace T3, T4, and T8 with AST-based parsers.

- **Library-doc freshness** · Library-doc freshness is best-effort.

If Context7 MCP, primary URL, fallback URL, and GitHub release API all fail (a network fully offline environment), the fetcher writes OFFLINE.md and continues. The child orchestrator then operates with stale or absent documentation, increasing AIE-002 (fabricated dependency) risk to ~30–40%. Confidence in this risk estimate: ~70%.

- **AIE catalog drift** · Auto-extension of the AIE catalog can drift.

Although HITL approval is required, low-vigilance approvals over many sessions may produce a catalog with semantically-overlapping entries. A periodic deduplication pass is recommended at the project level but is not yet implemented in the generator.

- **Regulatory currency** · EU AI Act mapping reflects the regulation as of 2026.

Implementing acts and harmonised standards under AI Act Article 40 may extend mandatory Article coverage. The mapping document (references/eu_ai_act_mapping.md) requires periodic review against the latest AESIA implementation guides.

- **Interview length** · The cover-the-bases interview produces a thorough but lengthy onboarding.

Thirty questions across eight phases is comprehensive but may be too much for low-stakes proof-of-concept work. A future version may add a fast-track interview (8 questions, minimal-risk default) gated behind explicit confirmation that the system is non-high-risk.

8.3 Need for Domain-Specific Review and Validation

system-designer is intended as a starting point, not a finished compliance solution. The following domains warrant additional validation before production use:

Domain	Additional review required	Confidence in default coverage
healthcare (clinical decision support)	MDR 2017/745 conformity assessment, ISO 13485, IRB / ethics committee review, TRIPOD-AI checklist sign-off, hospital-specific information-security review	~70%
fintech (credit / fraud)	DORA 2022/2554 ICT-risk assessment, PSD2/PSD3 mapping, model-risk-management review (SR 11-7 in US, SS1/23 in UK), explainability evidence per Art. 13	~75%
legal / public sector	GDPR DPIA, national e-government regulations, accessibility (WCAG 2.2 / EN 301 549), national language requirements, public-procurement transparency rules	~70%
research (with human subjects)	Helsinki Declaration alignment, IRB approval, GDPR Art. 89, consortium-specific data-sharing agreements	~80%
safety-critical (autonomous systems, robotics)	ISO 26262 (automotive), IEC 61508 (functional safety), ISO/PAS 21448 SOTIF, additional Article 9 rigour, near-miss + incident escalation drills	~60%
education (high-stakes assessment)	GDPR Art. 22 (automated decision-making), local pedagogical regulations, anti-discrimination tests by demographic subgroup	~70%

The confidence percentages reflect: (a) coverage of the relevant Article provisions in the audit baseline, and (b) the quality of cross-references emitted by the EU AI Act mapper. They do NOT reflect domain expertise. A qualified domain expert must review the audit sheet and the architecture document before any production deployment.

> *system-designer's audit baseline is a floor, not a ceiling. The user is responsible for extending the audit sheet with domain-specific rows and for engaging qualified human auditors before any high-risk deployment.*

9. Final Conclusion

system-designer makes the disciplined parts of AI-driven software development (specification, tracking, calibrated planning, EU AI Act audit scaffolding, error learning, human review) cheap to start and hard to abandon. It does this by being a generator rather than a system, by stopping at hand-off rather than running indefinitely, and by embedding five mutually reinforcing principles — portability, calibration, on-demand generation, prompt-architect dependency, and living documentation — into every emitted artifact.

The deliverable from any invocation is not a finished compliant system. It is a credible starting point that respects the regulatory baseline, exposes uncertainty honestly, and leaves the human in charge at the moments that matter. The remaining work — domain validation, audit completion, and engineering execution — is what the child orchestrator and its human users are designed to do.

Confidence in this conclusion: ~85%. Residual uncertainty is concentrated in domain-specific validation gaps (Section 7.3) and in the unmodelled costs of human review at the two inviolable gates.

10. References

EU AI Act 2024/1689 (Regulation (EU) 2024/1689 of the European Parliament and of the Council of 13 June 2024 laying down harmonised rules on artificial intelligence). Official Journal of the European Union. eur-lex.europa.eu/eli/reg/2024/1689/oj.

AESIA — Spanish AI Supervisory Agency. Implementation guides 01–13 for Regulation 2024/1689 (2024–2025).

TRIPOD-AI Statement. Collins, G.S., et al. (2024). TRIPOD+AI: updated guidance for reporting clinical prediction models that use regression or machine learning methods. BMJ, 385, e078378.

CONSORT-AI Extension. Liu, X., et al. (2020). Reporting guidelines for clinical trial reports for interventions involving artificial intelligence: the CONSORT-AI extension. Nature Medicine, 26.

STARD-AI. Sounderajah, V., et al. (2020). Developing a reporting guideline for artificial intelligence-centred diagnostic test accuracy studies: the STARD-AI protocol. BMJ Open, 10.

SPIRIT-AI Extension. Cruz Rivera, S., et al. (2020). Guidelines for clinical trial protocols for interventions involving artificial intelligence: the SPIRIT-AI extension. Nature Medicine, 26.

ISO/IEC 42001:2023. Artificial intelligence management system. International Organization for Standardization.

ISO 13485:2016. Medical devices — Quality management systems.

MDR 2017/745. Regulation (EU) 2017/745 on medical devices.

DORA 2022/2554. Regulation (EU) 2022/2554 on digital operational resilience for the financial sector.

Anthropic prompt-engineering best practices. Anthropic course material referenced in Informes_Cursos_Anthropic/ within the system-designer repository.

system-designer source code. github.com/Diego-M-C/system-designer (v0.1.0, MIT License).